



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And
Exploitation Techniques(Sensitive Data Exposure)

Student Name: Abubakari-Sadique Hamidu

Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP,
CISM)

Lab 10: Vulnerable Component

1. Introduction

This lab demonstrates the security risks of using **outdated software components** in web applications. An old version of jQuery (v1.3.2), which contains known security weaknesses, was intentionally used to show how attackers can exploit such components. The lab also demonstrates how updating the component and fixing insecure code mitigates the vulnerability.

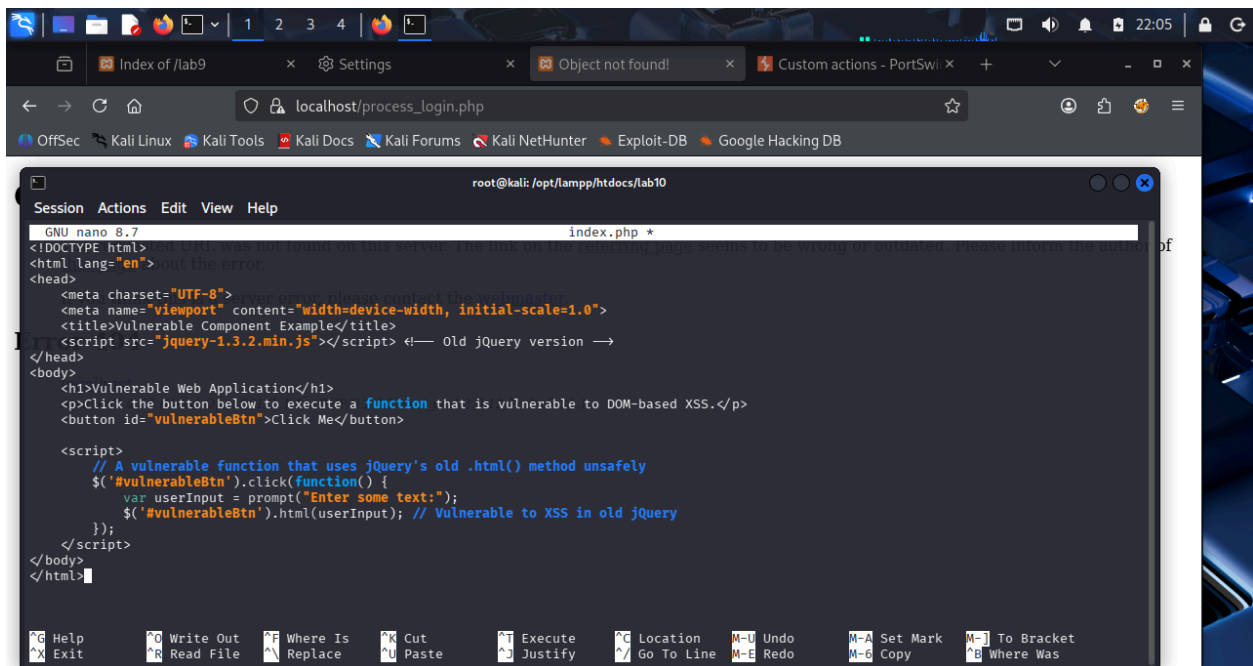
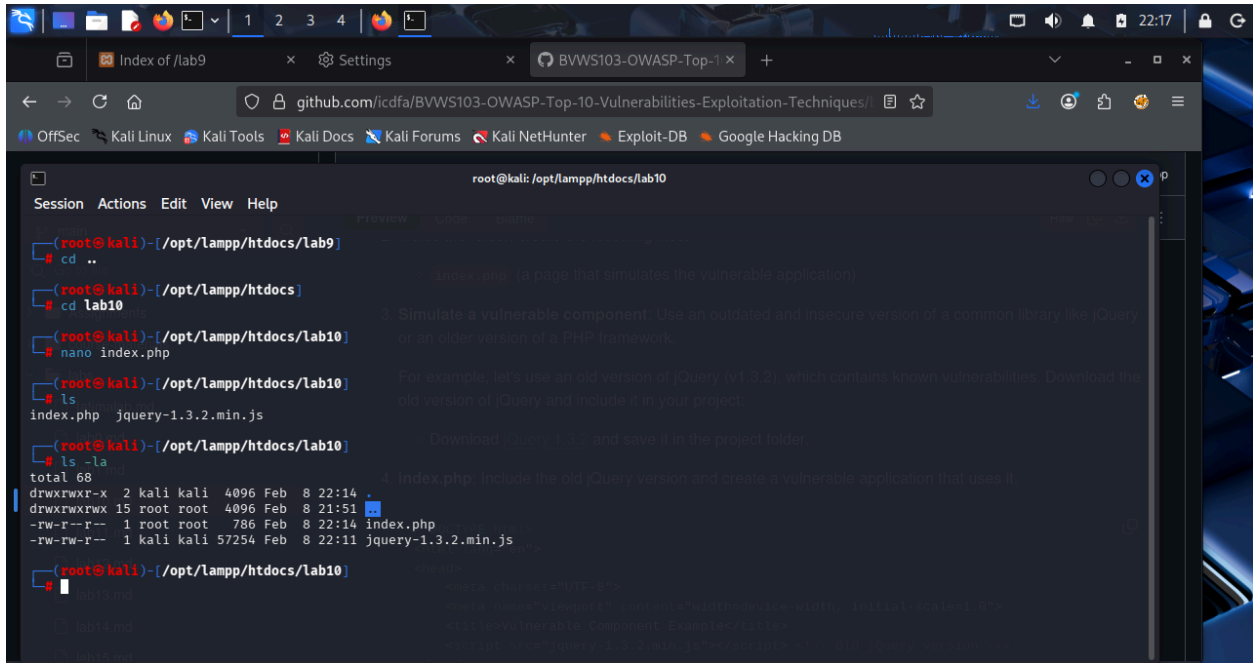
2. Environment Setup

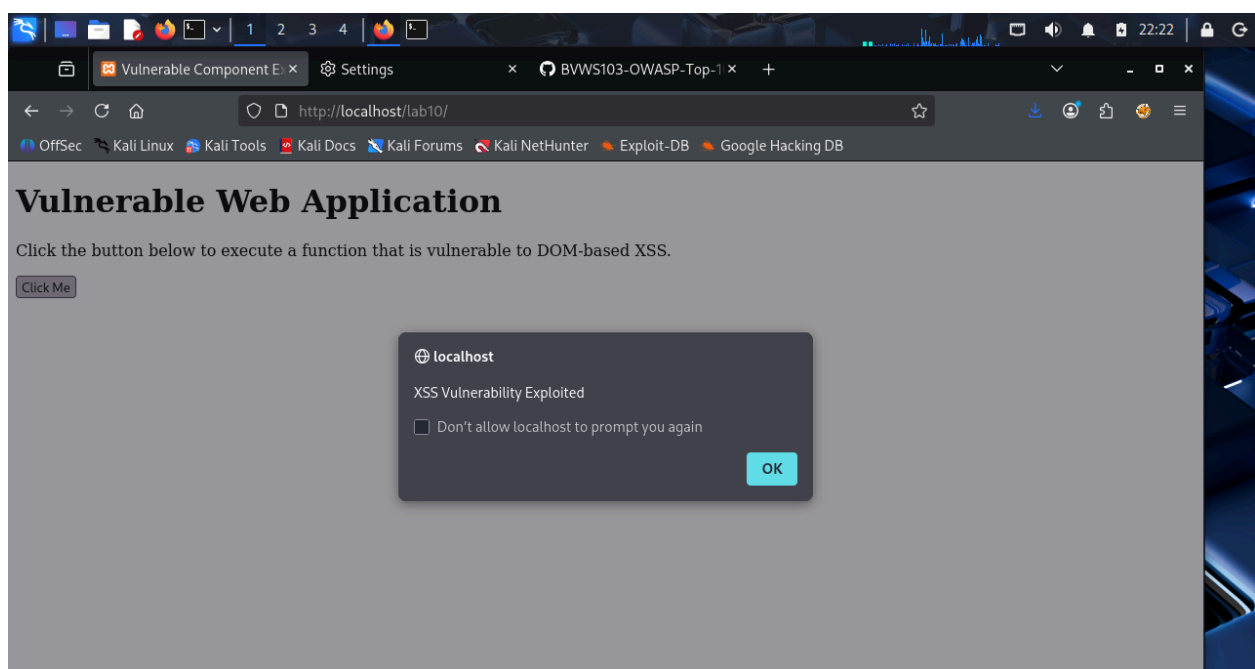
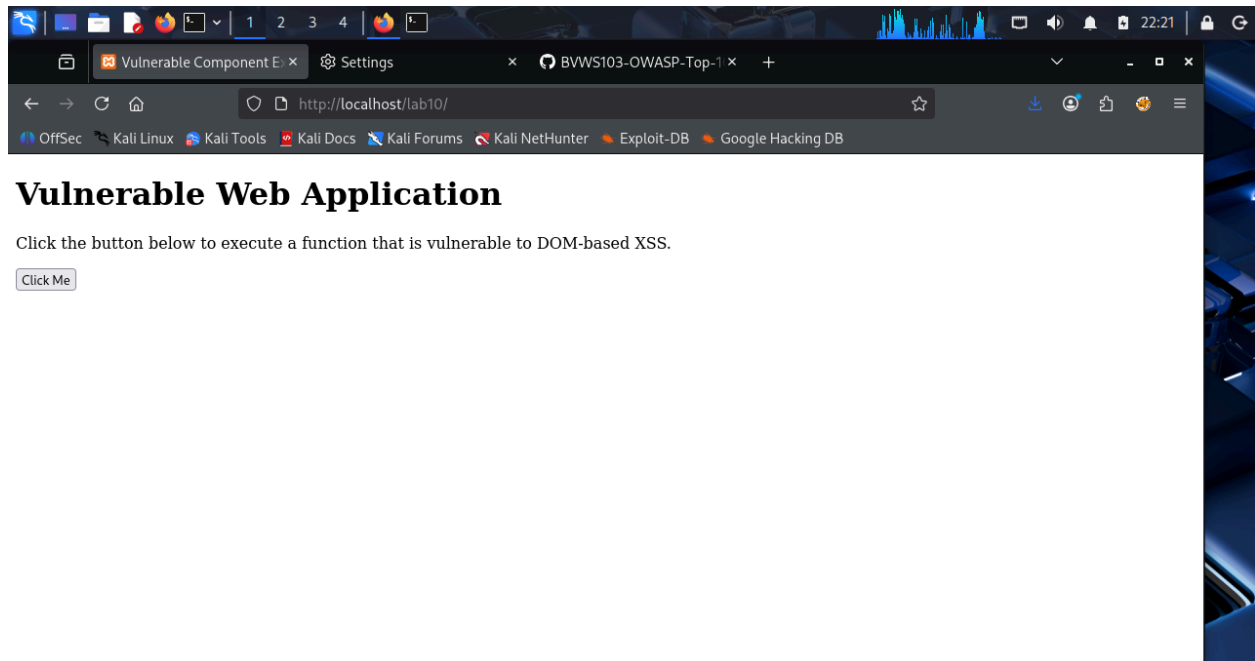
A local web server environment was set up using XAMPP/LAMPP. Apache was started successfully, and the lab files were placed inside the `htdocs` directory.

3. Exploiting the Vulnerable Component

An outdated version of jQuery (v1.3.2) was included in the application. The application used jQuery's `.html()` method to insert user input directly into the page without validation.

When a malicious script payload was entered, the browser executed the script, confirming a **DOM-based XSS vulnerability**.





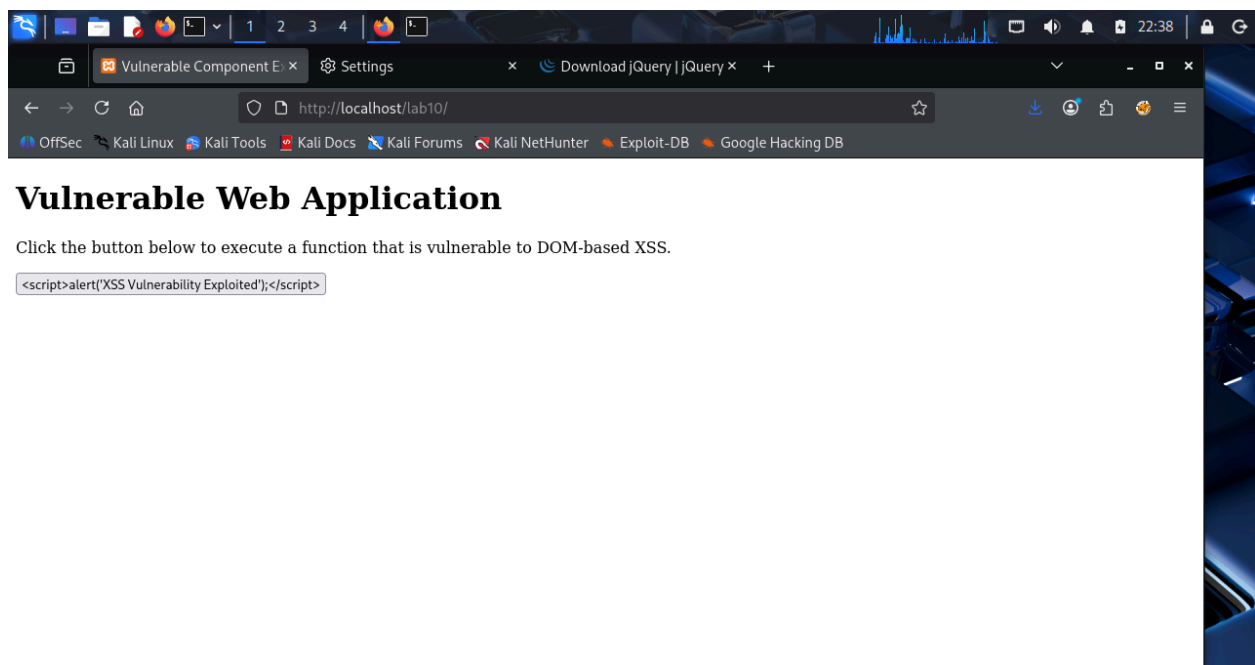
4. Mitigation of the Vulnerability

To fix the issue, the vulnerable component was updated to a modern version of jQuery (v3.6.0). Additionally, the insecure `.html()` method was replaced with `.text()` to safely handle user input.

After applying these changes, the same malicious payload no longer executed.

```
GNU nano 8.7 index.php *
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Vulnerable Component Example</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script> <!-- Updated jQuery version -->
</head>
<body>
  <h1>Vulnerable Web Application</h1>
  <p>Click the button below to execute a function that is vulnerable to DOM-based XSS.</p>
  <button id="vulnerableBtn">Click Me</button>

  <script>
    $('#vulnerableBtn').click(function() {
      var userInput = prompt("Enter some text:");
      // Use text() instead of html() to prevent XSS
      $('#vulnerableBtn').text(userInput); // Safely insert text without executing it
    });
  </script>
</body>
</html>
```



5. Conclusion

This lab shows that using outdated components poses serious security risks. Attackers can easily exploit known vulnerabilities in old libraries. Keeping software components updated and

following secure coding practices is essential to prevent attacks such as XSS and to maintain application security.